NLP LAB EXPERIMENTS

NAME: V Sunil Kumar

Reg no:192425292

COURSE CODE: DSA0311

COURSE NAME: NATURAL LANGUAGE PROCESSING

1. Write program demonstrates how to use regular expressions in Python to match and search for patterns in text.

Aim: To demonstrate how to use regular expressions in Python to match and search for patterns in text.

Algorithm

1. Start.
2. Import the re library.
3. Read the input text.
4. Search and match the required pattern.
5. Display the result.
6. Stop.

CODE:

```
import re
text = "Hello Python! My number is 9876543210."
result = re.search("Python", text)
if result:
    print("Word found:", result.group())
result = re.match("Hello", text)
if result:
    print("Match found:", result.group())
numbers = re.findall(r"\d+", text)
print("Numbers found:", numbers)
```

OUTPUT:

```
Word found: Python
Match found: Hello
Numbers found: ['9876543210']
```

Result: Successfully matched and searched for patterns in text using Python.

2. Implement a basic finite state automaton that recognizes a specific language or pattern. In this example, we'll create a simple automaton to match strings ending with 'ab' using python.

Aim: To implement a basic finite state automaton that accepts strings ending with ab.

Algorithm

1. Start.
2. Read a string.
3. Check whether the string ends with ab.
4. If yes, print Accepted; otherwise, print Rejected.
5. Stop.

CODE:

```
text = input("Enter a string: ")
if text.endswith("ab"):
    print("String Accepted")
else:
    print("String Rejected")
```

OUTPUT:

```
Enter a string: aaab
String Accepted
```

RESULT: Successfully accepted strings ending with ab.

3. Write program demonstrates how to perform morphological analysis using the NLTK library in Python.

Aim: To perform morphological analysis using the NLTK library in Python.

Algorithm

1. Start.
2. Import the NLTK stemming and lemmatization libraries.
3. Define a list of words.
4. Apply stemming and lemmatization to each word.
5. Display the results.
6. Stop.

CODE:

```
import nltk
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
words = ["playing", "played", "studies", "running"]
for word in words:
    print("Word:", word)
    print("Stem:", stemmer.stem(word))
    print("Lemma:", lemmatizer.lemmatize(word, pos="v"))
    print()
```

OUTPUT:

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
Word: playing
Stem: play
Lemma: play

Word: played
Stem: play
Lemma: play

Word: studies
Stem: studi
Lemma: study

Word: running
Stem: run
Lemma: run
```

RESULT: Successfully performed stemming and lemmatization using NLTK.

4 Implement a finite-state machine for morphological parsing. In this example, we'll create a simple machine to generate plural forms of English nouns using python.

Aim: To implement a finite-state machine to generate plural forms of English nouns using Python.

Algorithm

1. Start.
2. Read a singular noun.

3. Check the ending of the noun.
4. Apply the appropriate pluralization rule.
5. Display the plural form.
6. Stop.

CODE:

```
word = input("Enter a noun: ")
if word.endswith(("s", "sh", "ch", "x", "z")):
    plural = word + "es"
elif word.endswith("y") and word[-2] not in "aeiou":
    plural = word[:-1] + "ies"
else:
    plural = word + "s"
print("Plural form:", plural)
```

OUTPUT:

```
Enter a noun: baby
Plural form: babies
```

RESULT: Successfully generated plural forms of English nouns.

5 Use the Porter Stemmer algorithm to perform word stemming on a list of words using python libraries.

Aim: To perform word stemming on a list of words using the Porter Stemmer algorithm.

Algorithm

1. Start.
2. Import the Porter Stemmer library.
3. Define a list of words.
4. Apply the Porter Stemmer to each word.
5. Display each word and its stem.
6. Stop.

CODE:

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()
```

```
words = ["playing", "played", "plays", "running", "happily"]

for word in words:
    print(word, "->", ps.stem(word))
```

OUTPUT

```
playing -> play
played -> play
plays -> play
running -> run
happily -> happili
```

RESULT: Successfully reduced words to their stems using the Porter Stemmer algorithm.

6 Implement a basic N-gram model for text generation. For example, generate text using a bigram model using python.

Aim: To implement a basic N-gram model for text generation using a bigram model in Python.

Algorithm

1. Start.
2. Read the input text.
3. Split the text into words.
4. Form pairs of consecutive words (bigrams).
5. Generate a word sequence using the bigrams.
6. Display the generated text.
7. Stop.

CODE

```
import random

words = "I like Python I like coding".split()

word = "I"

result = [word]

for i in range(4):
    next_words = [words[j+1] for j in range(len(words)-1) if words[j] == word]
    if not next_words:
        break
    word = random.choice(next_words)
    result.append(word)
```

print("Generated text:", " ".join(result))

OUTPUT

```
Generated text: I like Python I like
```

RESULT: The program builds a bigram model from the given text and generates new text by predicting the next word based on the current word.

7 Write program using the NLTK library to perform part-of-speech tagging on a text.

Aim: To perform part-of-speech tagging on a text using the NLTK library in Python.

Algorithm

1. Start.
2. Import the NLTK library.
3. Read the input text.
4. Tokenize the text into words.
5. Assign a part-of-speech tag to each word.
6. Display the words and their tags.
7. Stop.

CODE

import nltk

nltk.download('punkt')

nltk.download('punkt_tab')

nltk.download('averaged_perceptron_tagger_eng')

text = "The cat runs fast."

words = nltk.word_tokenize(text)

tags = nltk.pos_tag(words)

print(tags)

OUTPUT:

```
[('The', 'DT'), ('cat', 'NN'), ('runs', 'VBZ'), ('fast', 'RB'), ('.', '.')]
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
```

RESULT: The program successfully identifies the parts of speech of words in the given text.

8 Implement a simple stochastic part-of-speech tagging algorithm using a basic probabilistic model to assign POS tags using python.

Aim: To implement a simple stochastic POS tagging algorithm using a basic probabilistic model in Python.

Algorithm

1. Start.
2. Define a list of words and possible POS tags.
3. Assign a tag to each word using a probabilistic model.
4. Display each word with its assigned tag.
5. Stop.

CODE

```
import random

words = ["I", "love", "Python"]

tags = ["PRP", "VBP", "NN"]

for word in words:
    tag = random.choice(tags)
    print(word, ":", tag)
```

OUTPUT

```
I : PRP
love : NN
Python : PRP
```

Result: The program demonstrates assigning POS tags using random selection as a simple probabilistic approach.

9 Implement a rule-based part-of-speech tagging system using regular expressions using python.

Aim: To implement a rule-based part-of-speech tagging system using regular expressions in Python.

Algorithm

1. Start.
2. Import the re library.

3. Read the input text.
4. Split the text into words.
5. Apply regular-expression rules to identify word types.
6. Display each word and its tag.
7. Stop.

CODE

```
import re

words = "The cat runs quickly"

for word in words.split():
    if re.match(r".*ly$", word):
        tag = "Adverb"
    elif re.match(r".*ing$", word):
        tag = "Verb"
    elif word in ["cat", "dog"]:
        tag = "Noun"
    else:
        tag = "Other"
    print(word, "-", tag)
```

OUTPUT:

```
The - Other
cat - Noun
runs - Other
quickly - Adverb
```

Result: The program uses regular-expression rules to assign basic parts-of-speech tags to words.

10 Implement transformation-based tagging using a set of transformation rules, apply a simple rule to tag words using python.

Aim: To implement transformation-based tagging using a simple set of rules in Python.

Algorithm

1. Start.
2. Define a list of words.
3. Assign initial tags to the words.

4. Apply transformation rules to change the tags.
5. Display the final tags.
6. Stop.

CODE:

```
words = ["I", "play", "playing", "happy"]
tags = ["Noun", "Noun", "Noun", "Noun"]
for i in range(len(words)):
    if words[i].endswith("ing"):
        tags[i] = "Verb"
    elif words[i] == "I":
        tags[i] = "Pronoun"
for word, tag in zip(words, tags):
    print(word, "-", tag)
```

OUTPUT:

```
I - Pronoun
play - Noun
playing - Verb
happy - Noun
```

Result: The program applies transformation rules to change the initial tags and assign suitable tags to words.

11 Implement a simple top-down parser for context-free grammars using python.

Aim: To implement a simple top-down parser for a context-free grammar using Python.

Algorithm

1. Start.
2. Define a simple grammar.
3. Read a sentence.
4. Split the sentence into words.
5. Check whether the sentence matches the grammar.
6. Display Accepted or Rejected.
7. Stop.

CODE

```
def parse(sentence):
    words = sentence.split()
    if len(words) == 2 and words[0] == "the" and words[1] == "cat":
        print("Sentence Accepted")
    else:
        print("Sentence Rejected")
sentence = input("Enter a sentence: ")
parse(sentence)
```

OUTPUT

```
Enter a sentence: the cat
Sentence Accepted
```

Result: The program checks whether a sentence matches the simple grammar pattern the cat.

12 Implement an Earley parser for context-free grammars using a simple python program.

Aim: To implement a simple Earley parser for a context-free grammar using Python.

Algorithm

1. Start.
2. Import the NLTK library.
3. Define a context-free grammar.
4. Create an Earley parser.
5. Read and tokenize a sentence.
6. Parse the sentence using the grammar.
7. Display Accepted or Rejected.
8. Stop.

CODE

```
import nltk
grammar = nltk.CFG.fromstring("""
S -> NP VP | NP
NP -> Det N
```

```
Det -> 'the' | 'The'
N -> 'cat' | 'dog'
VP -> 'runs' | 'plays'
""")
parser = nltk.EarleyChartParser(grammar)
sentence = input("Enter a sentence: ").split()
try:
    if any(parser.parse(sentence)):
        print("Accepted")
    else:
        print("Rejected")
except ValueError as e:
    print("Error:", e)
    print("Rejected (Word not in grammar)")
```

OUTPUT

```
Enter a sentence: the dog
Accepted
```

Result: The program uses an Earley parser to check whether a sentence follows the given context-free grammar.

13 Generate a parse tree for a given sentence using a context-free grammar using python program.

Aim: To generate a parse tree for a given sentence using a context-free grammar in Python.

Algorithm

1. Start.
2. Define a context-free grammar.
3. Create a chart parser.
4. Read a sentence and tokenize it.
5. Generate the parse tree.
6. Display the parse tree.
7. Stop.

CODE

```
import nltk
grammar = nltk.CFG.fromstring("""
S -> NP VP
NP -> 'the' 'cat'
VP -> 'runs'
""")
parser = nltk.ChartParser(grammar)
sentence = "the cat runs".split()
for tree in parser.parse(sentence):
    print(tree)
    tree.pretty_print()
```

OUTPUT:

```
(S (NP the cat) (VP runs))
          S
      ____|___
     NP       VP
  ___|___     |
the     cat runs
```

Result: The program generates and displays a parse tree for the given sentence using a context-free grammar.

14 Create a program in python to check for agreement in sentences based on a context-free grammar's rules.

Aim: To check sentence agreement using context-free grammar rules in Python.

Algorithm

1. Start.
2. Read the subject and verb.
3. Check the subject–verb agreement rules.
4. If the subject and verb agree, display Correct.
5. Otherwise, display Incorrect.
6. Stop.

CODE

```
subject = input("Enter subject (He/They): ")
verb = input("Enter verb (run/runs): ")
```

```
if (subject == "He" and verb == "runs") or (subject == "They" and verb == "run"):
    print("Correct sentence")
else:
    print("Incorrect sentence")
```

OUTPUT

```
Enter subject (He/They): He
Enter verb (run/runs): runs
Correct sentence
```

Result: The program checks subject–verb agreement in a sentence.

15 Implement probabilistic context-free grammar parsing for a sentence using python.

Aim: To implement probabilistic context-free grammar parsing for a sentence using Python.

Algorithm

1. Start.
2. Import the NLTK library.
3. Define a probabilistic context-free grammar with probabilities.
4. Create a Viterbi parser.
5. Read and tokenize a sentence.
6. Parse the sentence and calculate its probability.
7. Display the parse tree and probability.
8. Stop.

CODE

```
import nltk
grammar = nltk.PCFG.fromstring("""
S -> NP VP [1.0]
NP -> 'the' 'cat' [1.0]
VP -> 'runs' [1.0]
""")
parser = nltk.ViterbiParser(grammar)
sentence = "the cat runs".split()
for tree in parser.parse(sentence):
```

```
print(tree)
```

OUTPUT

```
(S (NP the cat) (VP runs)) (p=1)
```

Result: The program identifies named entities such as persons, locations, and organizations in the given text.

16. Implement a Python program using the SpaCy library to perform Named Entity Recognition (NER) on a given text.

Aim: To identify named entities in a given text using the spaCy library in Python.

Algorithm

1. Start.
2. Import the spaCy library.
3. Load the English language model.
4. Read the input text.
5. Identify named entities in the text.
6. Display the entities and their labels.
7. Stop.

CODE

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "Ravi lives in Chennai and works at Google."

doc = nlp(text)

for ent in doc.ents:
    print(ent.text, "-", ent.label_)
```

OUTPUT

```
Chennai - ORG
Google - ORG
```

Result: The program identifies named entities such as persons, locations, and organizations in the given text.

17 Write program demonstrates how to access WordNet, a lexical database, to retrieve synsets and explore word meanings in python.

Aim: To retrieve synsets and explore word meanings using WordNet in Python.

Algorithm

1. Start.
2. Import WordNet from NLTK.
3. Read a word from the user.
4. Retrieve its synsets from WordNet.
5. Display the meanings of the word.
6. Stop.

CODE

```
from nltk.corpus import wordnet

word = input("Enter a word: ")

for syn in wordnet.synsets(word):

   print("Meaning:", syn.definition())
```

OUTPUT

```
Enter a word: happy
Meaning: enjoying or showing or marked by joy or pleasure
Meaning: marked by good fortune
Meaning: eagerly disposed to act or to be of service
Meaning: well expressed and to the point
```

Result: The program retrieves different meanings of a word from the WordNet lexical database.

18 Implement a simple FOPC parser for basic logical expressions using python program.

Aim: To implement a simple First-Order Predicate Calculus (FOPC) parser for basic logical expressions using Python.

Algorithm

1. Start.
2. Import the regular expression library.
3. Read a logical expression.
4. Check whether it follows the required predicate format.
5. Display Valid or Invalid.
6. Stop.

CODE

```
import re
```

```
expression = input("Enter a logical expression: ")
if re.match(r"^[A-Za-z]+\([A-Za-z]+\)$", expression):
    print("Valid logical expression")
else:
    print("Invalid logical expression")
```

OUTPUT

```
Enter a logical expression: Human(Ravi)
Valid logical expression
```

Result: The program checks whether a basic predicate expression follows the required format, such as Predicate(Constant).

19 Create a program for word sense disambiguation using the Lesk algorithm using python.

Aim: To identify the correct meaning of a word using the Lesk algorithm in Python.

Algorithm

1. Start.
2. Import the Lesk algorithm from NLTK.
3. Read a sentence containing an ambiguous word.
4. Identify the intended word.
5. Apply the Lesk algorithm using the sentence context.
6. Display the selected meaning.
7. Stop.

CODE

```
import nltk
from nltk.wsd import lesk
nltk.download('wordnet')
sentence = "I went to the bank to deposit money".split()
meaning = lesk(sentence, 'bank')
print("Word:", "bank")
print("Meaning:", meaning.definition() if meaning else "Meaning not found")
```

OUTPUT

```
Word: bank
Meaning: a container (usually with a slot in the top) for keeping money at home
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
```

Result: The program uses the Lesk algorithm to identify the appropriate meaning of a word based on its context.

20 Implement a basic information retrieval system using TF-IDF (Term Frequency-Inverse Document Frequency) for document ranking using python.

Aim: To implement a basic information retrieval system using TF-IDF to rank documents in Python.

Algorithm

1. Start.
2. Import the TF-IDF vectorizer and cosine similarity functions.
3. Define a collection of documents.
4. Read the search query.
5. Calculate TF-IDF vectors for documents and query.
6. Calculate similarity scores.
7. Rank documents by similarity.
8. Display the ranked documents.
9. Stop.

CODE

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
docs = ["Python is easy", "Python is powerful", "Java is popular"]
query = input("Enter search query: ")
vectorizer = TfidfVectorizer()
data = vectorizer.fit_transform(docs + [query])
scores = cosine_similarity(data[-1], data[:-1])[0]
for i in scores.argsort()[::-1]:
    print(docs[i], "-", round(scores[i], 2))
```

OUTPUT

```
Enter search query: Python
Python is powerful - 0.47
Python is easy - 0.47
Java is popular - 0.0
```

Result: The program ranks documents according to their similarity to the search query using TF-IDF and cosine similarity.